

SQL -DATA DEFINITION LANGUAGE

(Commands and Constraints)

Project Objective: The main objective of this project is to store and manage Employee Database using MySQL DDL commands to ensure data integrity and consistency.

Dataset Description: The given Dataset contains Three tables such as employees, departments and locations of the organization. Inside the table we have to create the Attributes with specific constraints.

Column Description:

Table:Departments

Column Name	Data Type
Department ID	It should be unique so it is Primary Key
Department Name	It contains Department Name, it is VARCHAR

Table:Location

Column Name	Data Type
Location ID	It should be unique so it is primary key
Location Name	It contains Location,it is VARCHAR

Table:Employees

Column Name	Data Type
Employee ID	It is Primary Key because it should be unique.
Employee Name	It contains name of the employee so it is VARCHAR
Gender	It should be either male or female,it is ENUM data type
Age	Age of the employee should be INT
Hire Date	Hiring date is in DATE data type
Salary	It is in decimal data type
Designation	It is in VARCHAR data type

In the employees table,Department Id and Location ID are **Foreign Key** because they are references from department and location tables.

DDL Commands:

a)Database and Table creation:

1.Create Database for this project:

Command Used:Create database if not exists employeedb;

Use employeedb;

Explanation:A database named employeedb is created when there is no database in that name previously in the schema The database is used to store the data.

```
-- Create database
create database if not exists employeedb;
use employeedb;
```

2.Create all three tables with appropriate data types and relationships:

Command Used:Create Table if not exists department()

Create Table if not exists location()

Create Table if not exists employee()

Explanation:Tables are created with given columns and specific data types.Unique values are given as primary key and some attributes are in Foreign key as references for other attributes.

Department Table:

```
4
5 • ○ create table if not exists department(
6     Department_id INT PRIMARY KEY,
7     Department_name VARCHAR(100)
8 );
9
```

Location Table:

```
9
10 • ○ create table if not exists locations(
11     Location_id INT PRIMARY KEY,
12     Location_name VARCHAR(30)
13 );
14
```

Employee Table:

```

create table if not exists employees(
  Employee_id INT PRIMARY KEY,
  Employee_name VARCHAR(50),
  Gender ENUM('M','F'),
  Age INT,
  Hire_date DATE,
  Designation VARCHAR(100),
  Department_id INT,
  Location_id INT,
  Salary DECIMAL(10,2),
  FOREIGN KEY(Department_id)REFERENCES department(Department_id),
  FOREIGN KEY(Location_id)REFERENCES location(Location_id)
);

```

b)Table Creation:

1.Add new column 'Email' to employees table:

Command Used:ALTER TABLE employees

MODIFY COLUMN Designation VARCHAR(200);

Explanation:Table is altered and modified using SQL Commands.

```

ALTER TABLE employees
ADD COLUMN Email VARCHAR(100);

```

2.Modify data type of 'designation' column in employee table:

Command used:ALTER TABLE employees

MODIFY COLUMN Designation VARCHAR(200);

Explanation:Designation column is modified from 100 characters to 200 characters.

```

ALTER TABLE employees
MODIFY COLUMN Designation VARCHAR(200);

```

3.Drop 'age' column from employee table:

Command Used:ALTER TABLE employees

DROP COLUMN Age;

```
RENAME COLUMN designation TO name(400)
ALTER TABLE employees
DROP COLUMN Age;
ALTER TABLE employees
```

4. Rename 'hire date' column to 'date of joining':

Command used: ALTER TABLE employees
RENAME COLUMN Hire_date TO Date_of_joining;

```
DROP COLUMN Age;
ALTER TABLE employees
RENAME COLUMN Hire_date TO Date_of_Joining;
ALTER TABLE employees
```

C. Table Renaming:

1. Rename 'Departments'table to 'Department_Info'

Command Used: ALTER TABLE employees
RENAME TO Departments_Info;

```
ALTER TABLE department
RENAME TO Departments_Info;
ALTER TABLE departments
```

2. Rename 'Location'table to 'Locations':

Command Used: ALTER TABLE location
RENAME TO Locations;

```
RENAME TO departments_info
ALTER TABLE location
RENAME TO Locations;
```

D. Table Truncation:

Command used: TRUNCATE TABLE Employees;

```
TRUNCATE TABLE Employees;
```

E. Drop Employee table and 'employee' database:

Command Used: DROP TABLE employees;
DROP DATABASE employeedb;

```
DROP TABLE employees;  
DROP DATABASE employeedb;
```

CONSTRAINTS:

a) Database Recreation:

1. Drop the 'Employee' database if it exists:

The database was dropped previously using DROP DATABASE command.

2. Recreate it using provided schema using appropriate constraints:

It is recreated using provided schema that all tables are created with appropriate constraints.

```
create table if not exists department(  
    Department_id INT PRIMARY KEY,  
    Department_name VARCHAR(100) NOT NULL  
);  
  
create table if not exists location(  
    Location_id INT AUTO_INCREMENT PRIMARY KEY,  
    Location_name VARCHAR(30) NOT NULL  
);  
  
create table if not exists employees(  
    Employee_id INT PRIMARY KEY,  
    Employee_name VARCHAR(50) NOT NULL,  
    Gender ENUM('M', 'F'),  
    Age INT CHECK(Age >= 18),  
    Hire_date DATE DEFAULT(current_date),  
    Designation VARCHAR(100),  
    Department_id INT,  
    Location_id INT.
```

3.Establish links between 'department id' and 'location id' in employees table:

```
FOREIGN KEY(Department_id)REFERENCES department(Department_id),  
FOREIGN KEY(Location_id)REFERENCES location(Location_id)|  
);
```

In employees table,both department and location id are Foreign keys which are references from other tables.

b)Departments Table:

'Department id' uniquely identifies each department in the table so it can be given as Primary Key,'Department Name' is set to not null constraints to avoid duplicate and null entries.

```
create table if not exists department(  
    Department_id INT PRIMARY KEY,  
    Department_name VARCHAR(100) NOT NULL  
);
```

c)Locations Table:

For location auto increment is given to increment sequentially to generate unique identifiers.

```
create table if not exists location(  
    Location_id INT AUTO_INCREMENT PRIMARY KEY,  
    Location_name VARCHAR(30) NOT NULL  
);
```

Not Null constraints are also given to location to prevent null and duplicate locations.

Result Grid Filter Rows: Export: Wrap Cell Content:					
Field	Type	Null	Key	Default	Extra
Location_id	int	NO	PRI	HULL	auto_increment
Location_name	varchar(30)	NO		HULL	

d)Employee Table:

1.Each employee has a distinct identifier using Primary Key and restriction to employee's name is provided using not null constraints.

```
);
```

```
create table if not exists employees(  
Employee_id INT PRIMARY KEY,  
Employee_name VARCHAR(50) NOT NULL,
```

2. 'Gender' field is limited only to 'M' and 'F' using Enum constraints. For Employee age conditions are set using Check as 18 or above.

```
Gender ENUM('M','F'),  
Age INT CHECK(Age>=18),
```

3.Current date is assigned to 'hire date' automatically if not dates not specified using default date function.

```
Hire_date DATE DEFAULT(current_date),  
Designation VARCHAR(100),
```

Conclusion:

The output of department,location and employees table are as follows:

Department Table:

Field	Type	Null	Key	Default	Extra
Department_id	int	NO	PRI	NONE	
Department_name	varchar(100)	NO		NONE	

Location Table:

Result Grid  Filter Rows: Export:  Wrap Cell Content: 					
Field	Type	Null	Key	Default	Extra
Location_id	int	NO	PRI	NULL	auto_increment
Location_name	varchar(30)	NO		NULL	

Employees Table:

Result Grid  Filter Rows: Export:  Wrap Cell Content: 					
Field	Type	Null	Key	Default	Extra
Employee_id	int	NO	PRI	NULL	
Employee_name	varchar(50)	NO		NULL	
Gender	enum('M','F')	YES		NULL	
Age	int	YES		NULL	
Hire_date	date	YES		curdate()	DEFAULT_GENERATED
Designation	varchar(100)	YES		NULL	
Department_id	int	YES	MUL	NULL	
Location_id	int	YES	MUL	NULL	
Salary	decimal(10,2)	YES		NULL	
Email	varchar(50)	YES		NULL	

Employee database is designed and managed using MYSQL DDL commands with specific constraints and output is executed.